

Mechanized Specifications (4): Other PLs and Non-PL Software

Sukyoung Ryu

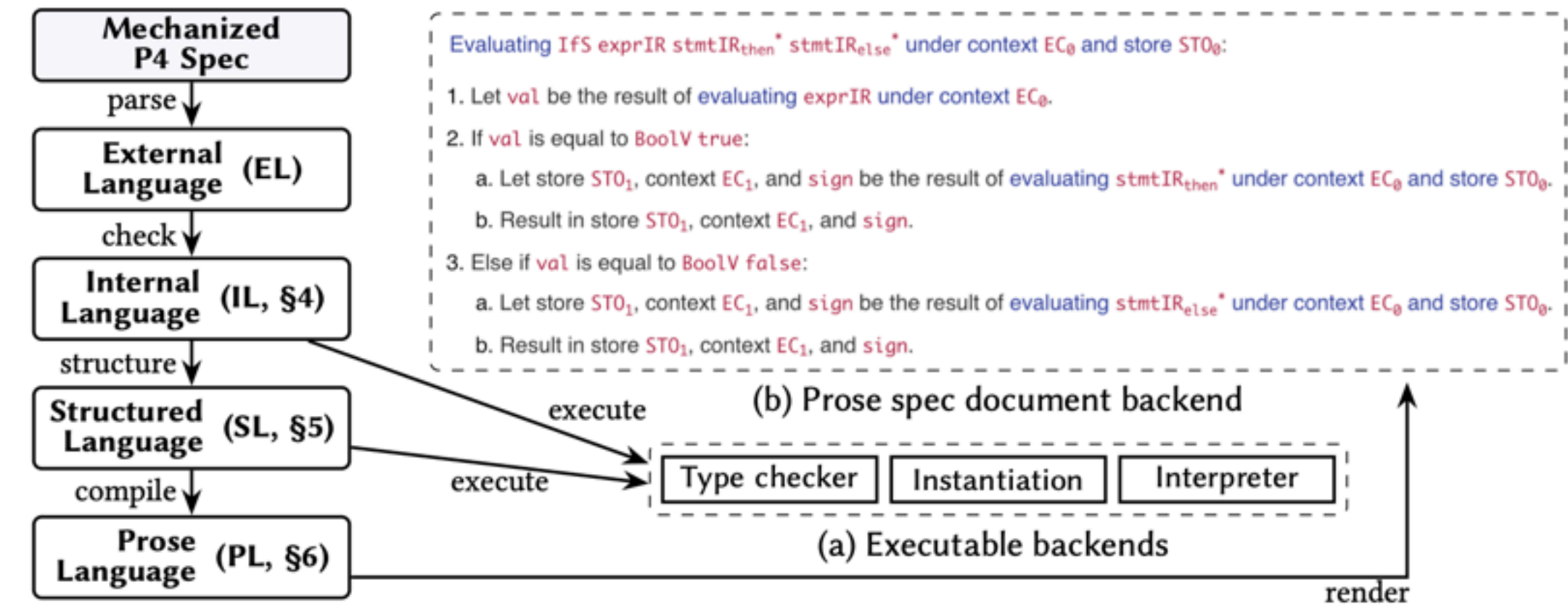
August 21, 2026

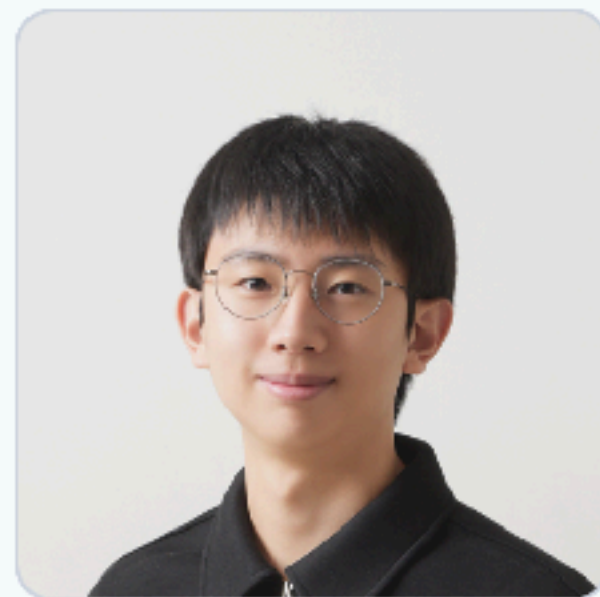
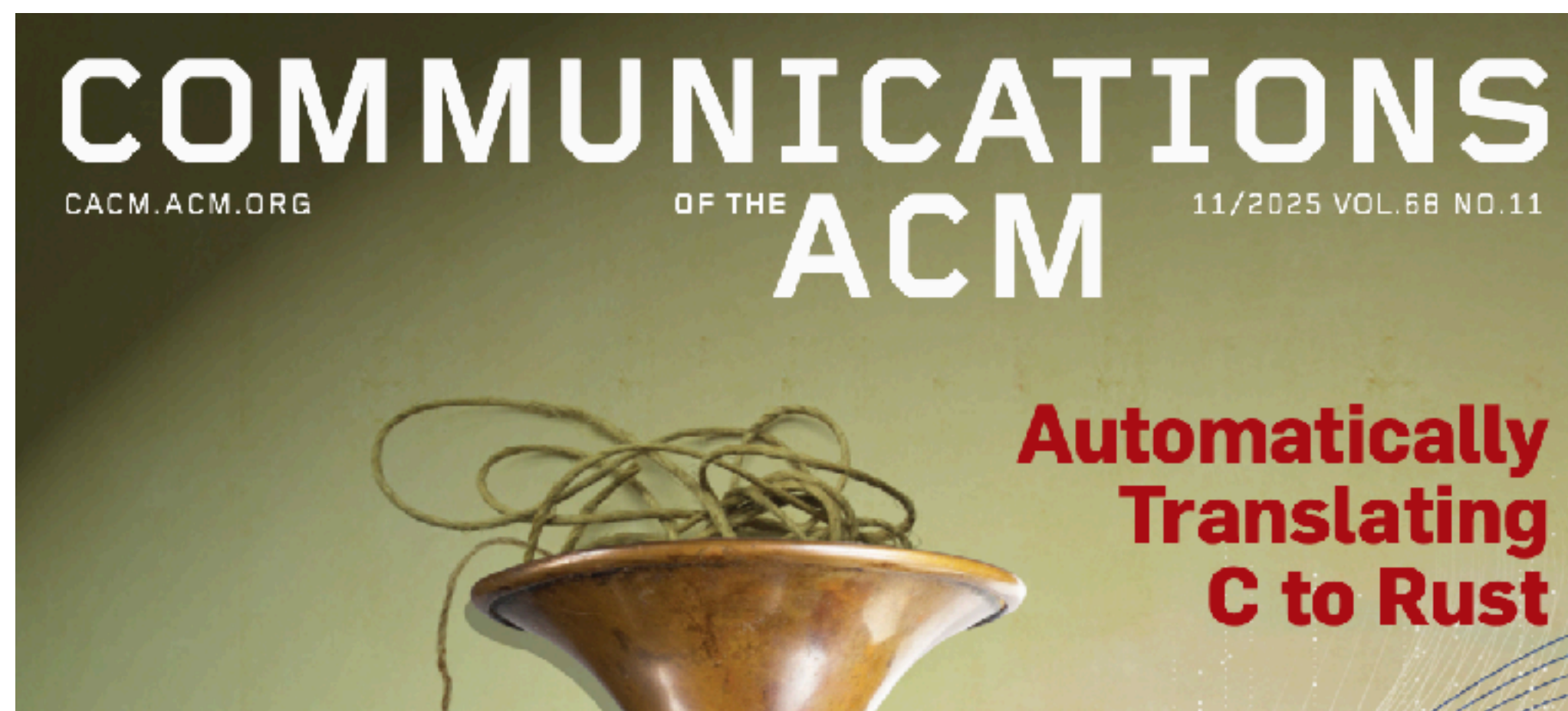
Mechanized Specifications

- **Lecture 1: JavaScript and ESMeta**
- **Lecture 2: WebAssembly and SpecTec**
- **Lecture 3: P4 and P4-SpecTec**
- **Lecture 4: Other PLs and Non-PL Software**

P4 and P4-SpecTec

- **January 2024** First meeting
- **October 2024** Presentation @ P4 Workshop
- **December 2024** First presentation @ P4 LDWG Meeting
- **March 2026** Conditional acceptance @ P4 LDWG meeting
- **July 2026** FSE paper
- **October 2026** OOPSLA paper
- **December 2026** Official acceptance @ P4 LDWG meeting (planned)





<https://plrg.unist.ac.kr/members/jaemin-hong.html>

Jaemin Hong

Assistant Professor

Department of Computer Science and Engineering, UNIST



research and advances



DOI:10.1145/3737696

Automatic C-to-Rust translation tools are helpful, but they produce unsafe and unidiomatic code. What can be done to address these issues?

BY JAEMIN HONG AND SUKYOUNG RYU

Automatically Translating C to Rust

IN THE SOFTWARE industry, legacy systems developed in older languages often evolve by being reimplemented in newer languages that offer modern language features. For example, Twitter migrated from Ruby to Scala to enhance performance and reliability;²⁹ Dropbox rewrote its Python backends in Go to leverage better concurrency support and faster execution;¹⁸ and banking systems originally written in Cobol have evolved to Java or C# for easier maintenance and integration with modern infrastructures.³

One of the most critical language migrations needed today is the shift from C to newer languages, which can improve the reliability of important system programs. Despite its popularity in systems programming, C is infamous for its limited language-level safety mechanisms. C programs are prone to memory bugs, such as buffer overflow and use-after-free, which can

lead to severe security vulnerabilities—exemplified by the Heartbleed bug^a in OpenSSL. A large portion of vulnerabilities in legacy systems arise from memory bugs; for example, approximately 70% of the vulnerabilities in Microsoft's codebase are due to memory bugs.²⁸ Acknowledging these risks, even the White House recently recommended discouraging the use of C.²⁴

The most promising migration target for C is Rust,³¹ which provides strong safety guarantees while still allowing low-level memory control and high performance. Its safety guarantees are enabled by its ownership type system, which ensures the absence of memory bugs in programs that pass type checking.¹⁵ Due to this advantage, Rust has been widely adopted in systems programming, as demonstrated by the development of clean-slate system programs such as garbage collectors,²⁰ Web browsers,⁷ and operating systems.^{2,6,19,22}

Recognizing Rust's potential, the industry has shown significant interest in migrating legacy systems from C to Rust. Such migration allows developers to detect previously unknown

^a <https://heartbleed.com/>

» key insights

- Migrating legacy systems developed in C to Rust is a promising way to enhance reliability, thanks to Rust's strong safety guarantees.
- Automatic translators can facilitate migration from C to Rust, but existing translators generate unsatisfactory code by relying on language features whose safety is not validated by the compiler and code patterns considered unidiomatic by Rust developers.
- Carefully designed static analyses and code transformations can improve automatic translation by replacing unsafe features and unidiomatic patterns with safe and idiomatic alternatives.
- Although LLMs are another promising approach to C-to-Rust translation, they often produce code with type errors or behavior different from the original code. Combining LLMs with static analyses is a potential future research direction.

ILLUSTRATION BY VICTOR KUEN

Figure 1. Improving C2Rust-translated code by replacing unsafe features and unidiomatic patterns.

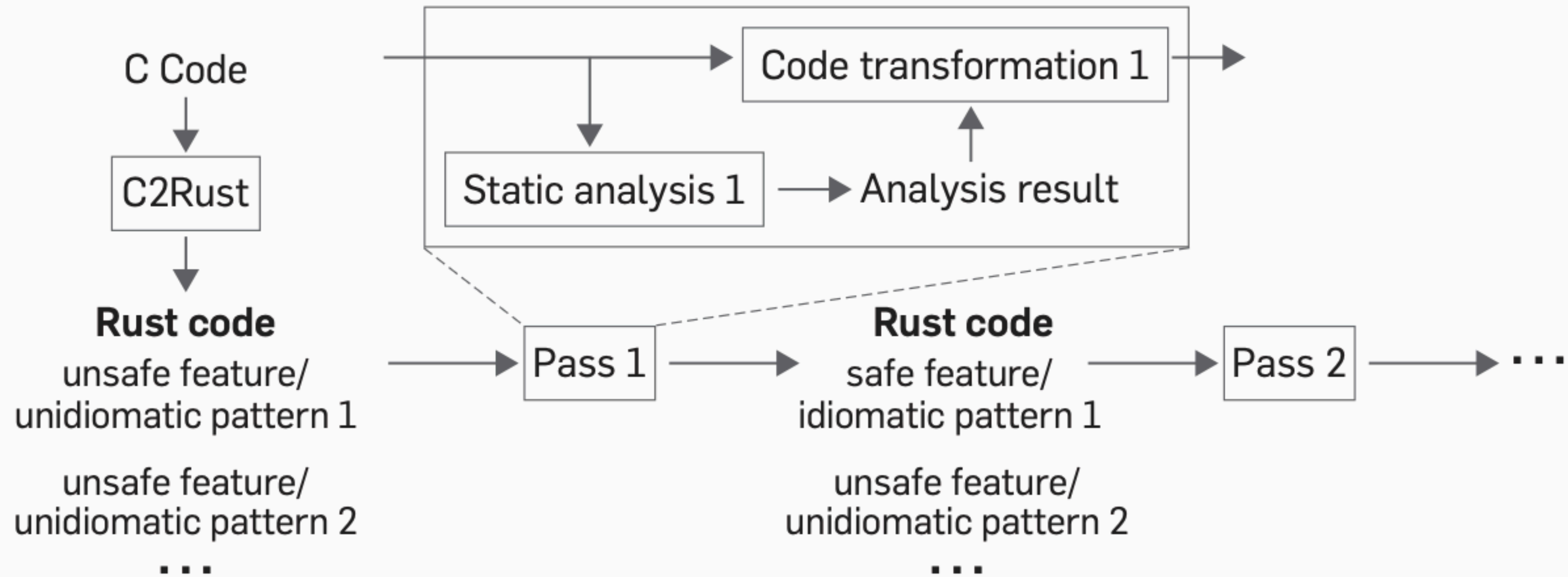
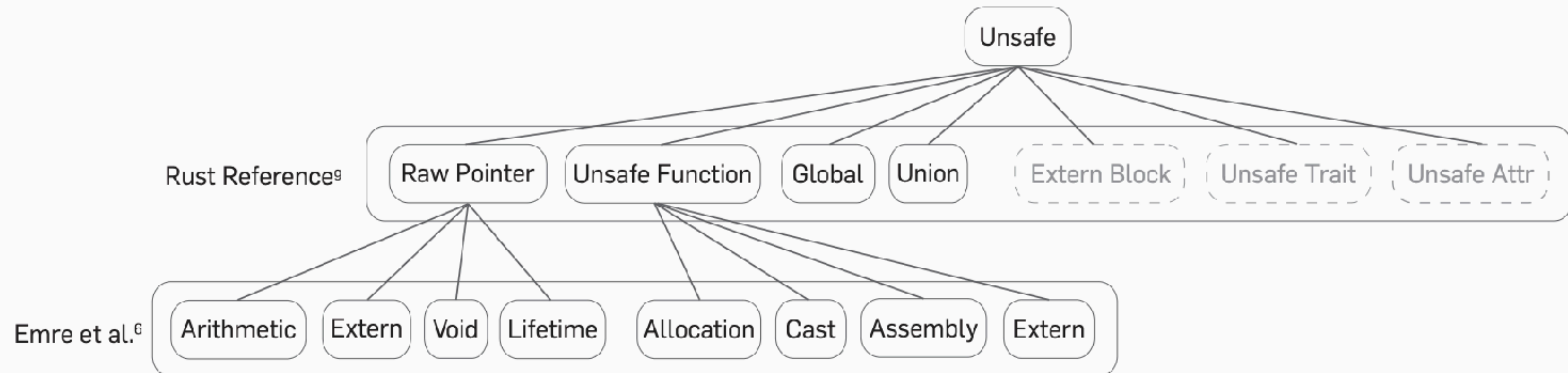


Table. List of unsafe features in C2Rust-translated code.

Feature	Relevant Categories	Addressed
Scalar pointer	Lifetime pointer, allocation function	✓
Array pointer	Arithmetic pointer, allocation function, extern function	✗
Lock	Extern pointer, extern function, global	✓
File	Extern pointer, extern function	✗
Subprocess	Extern function	✗
Union with tag	Union	✓
Void pointer argument	Void pointer, cast	✗

Figure 2. Classification of unsafe features in C2Rust-translated code.



The Fortress Language Specification

Version 1.0 β

Eric Allen
David Chase
Joe Hallett
Victor Luchangco
Jan-Willem Maessen
Sukyoung Ryu
Guy L. Steele Jr.
Sam Tobin-Hochstadt

Additional contributors:
Joao Dias
Carl Eastlund
Christine Flood
Yossi Lev
Cheryl McCosh

© Sun Microsystems, Inc.

March 6, 2007

Environments

$\Delta ::= \overrightarrow{\alpha <: N}$ bound environment
 $\Gamma ::= \overrightarrow{x : \tau}$ type environment

Program typing: $\vdash p : \tau$

[T-PROGRAM]
$$\frac{p = \overrightarrow{d} \ e \quad p; \emptyset; \emptyset \vdash \overrightarrow{d} \text{ ok} \quad p; \emptyset; \emptyset \vdash e : \tau \quad \text{acyclic}(\overrightarrow{d})}{\vdash p : \tau}$$

Acyclic type hierarchy: $\text{acyclic}(\overrightarrow{d})$

trait $T[\overrightarrow{\alpha} \text{ extends } \overrightarrow{N}] \text{ extends } \{\overrightarrow{M}\} \text{ end} \in \overrightarrow{d} \quad 1 \leq i \leq |\overrightarrow{\alpha}|$
(1) $M_i \neq T[-]$ implies $p; - \vdash [\overrightarrow{\tau} / \overrightarrow{\alpha}] \overrightarrow{M}_i \not\prec: T[\overrightarrow{\tau}]$
(The type names excluding self-extensions form an acyclic hierarchy.)
(2) $M_i = T[-]$ implies $M_j \neq T[-]$ $1 \leq j \leq |\overrightarrow{M}| \quad i \neq j$
(3) $M_i = T[\overrightarrow{\tau}]$ implies $(\tau_i = \alpha_i) \vee (\tau_i = N_i) \vee (\tau_i = \alpha_j \wedge \alpha_i = N_j) \quad 1 \leq j \leq |\overrightarrow{\alpha}| \quad i \neq j$
[ACYCLIC]
$$\text{acyclic}(\overrightarrow{d})$$

Definition typing: $p; \emptyset; \emptyset \vdash d \text{ ok}$

[T-TRAITDEF]
$$\frac{\Delta = \overrightarrow{\alpha <: N} \quad p; \Delta \vdash \overrightarrow{N} \text{ ok} \quad p; \Delta \vdash \overrightarrow{M} \text{ ok} \quad p; \Delta; \text{self} : T[\overrightarrow{\alpha}] \vdash T \text{ ok} \overrightarrow{md} \quad p = \text{oneOwner}(T)}{p; \emptyset; \emptyset \vdash \text{trait } T[\overrightarrow{\alpha} \text{ extends } \overrightarrow{N}] \text{ extends } \{\overrightarrow{M}\} \overrightarrow{md} \text{ end ok}}$$

[T-OBJECTDEF]
$$\frac{\overrightarrow{vd} = \overrightarrow{x' : \tau' = e} \quad \Delta = \overrightarrow{\alpha <: N} \quad p; \Delta \vdash \overrightarrow{N} \text{ ok} \quad p; \Delta \vdash \overrightarrow{\tau} \text{ ok} \quad p; \Delta \vdash \overrightarrow{M} \text{ ok} \quad p; \Delta; \overrightarrow{x} : \overrightarrow{\tau} \quad x'_1 : \tau'_1 \dots x'_{i-1} : \tau'_{i-1} \vdash x'_i : \tau'_i = e_i \text{ ok} \quad 1 \leq i \leq n \quad p; \Delta; \text{self} : O[\overrightarrow{\alpha}] \quad \overrightarrow{x} : \overrightarrow{\tau} \quad x' : \overrightarrow{\tau'} \vdash O \text{ ok} \overrightarrow{md} \quad p \vdash \text{oneOwner}(O)}{p; \emptyset; \emptyset \vdash \text{object } O[\overrightarrow{\alpha} \text{ extends } \overrightarrow{N}] (\overrightarrow{x} : \overrightarrow{\tau}) \text{ extends } \{\overrightarrow{M}\} \overrightarrow{md} \text{ end ok}}$$

One owner for all the visible methods: $p \vdash \text{oneOwner}(C)$

[ONEOWNER]
$$\frac{\forall m \in \text{visible}_p(C) . f \text{ only occurs once in } \text{visible}_p(C)}{p \vdash \text{oneOwner}(C)}$$

Method typing: $p; \Delta; \Gamma \vdash C \text{ ok} \overrightarrow{md}$

[T-METHODDEF]
$$\frac{- C[\overrightarrow{\alpha'} \text{ extends } -] \text{ extends } \{\overrightarrow{K}\} \in p \quad p = \text{override}(m, \{\overrightarrow{M}\}, [\overrightarrow{\alpha} \text{ extends } \overrightarrow{N}]) \quad \overrightarrow{\tau} \rightarrow \tau_0 \quad \Delta' = \Delta \ \alpha <: N \quad p; \Delta' \vdash \overrightarrow{N} \text{ ok} \quad p; \Delta' \vdash \overrightarrow{\tau} \text{ ok} \quad p; \Delta' \vdash \tau_0 \text{ ok} \quad p; \Delta'; \Gamma \quad \overrightarrow{x} : \overrightarrow{\tau} \vdash e : \tau' \quad p; \Delta' \vdash \tau' <: \tau_0}{p; \Delta; \Gamma \vdash C \text{ ok} \overrightarrow{md} [\overrightarrow{\alpha} \text{ extends } \overrightarrow{N}] (\overrightarrow{x} : \overrightarrow{\tau}) : \tau_0 = e}$$

Figure A.21: Static Semantics of Acyclic Core Fortress with Field Definitions (I)

Chapter 37

Algebraic Constraints

The traits in this component are used to describe properties of traits and their associated operators. These traits provide very few concrete methods, but specify abstract methods and property declarations.

37.1 Predicates and Equivalence Relations

A *predicate* is an operator that produces a boolean result. A binary predicate may be identified with a mathematical relation, where the predicate returns *true* in exactly those cases that its two operands satisfy the relation; therefore we use the mathematical terminology usually associated with relations to describe the properties of binary predicates.

```
trait UnaryPredicate[T extends UnaryPredicate[T, ~], opr ~]
  extends Any
  abstract opr ~(self): Boolean
end
```

A unary predicate is a prefix operator that takes one argument and returns a boolean value (*true* or *false*). Note that the symbol $\sim$ is a static parameter, used here as a “variable” name for an operator.

```
trait BinaryPredicate[T extends BinaryPredicate[T, ~], opr ~]
  extends Any
  abstract opr ~(self, other: T): Boolean
end
```

A binary predicate is an infix operator that takes two arguments and returns a boolean value. Thus, for example, any trait *T* that extends `BinaryPredicate[T,  $\sqsubset$ ]` necessarily has an infix method for the operator $\sqsubset$, and that operator returns a boolean value.

```
trait Reflexive[T extends Reflexive[T, ~], opr ~]
  extends { BinaryPredicate[T, ~] }
  property  $\forall(a: T) (a \sim a)$ 
end
```

A *reflexive* predicate always returns *true* when its operands are the same. Because this fact is expressed as a property declaration, the behavior of such an operator can be checked for correctness by unit testing.

```
trait PartialOrderAndMeetBoundedLattice[T extends PartialOrderAndMeetBoundedLattice[T,  $\preceq$ ,  $\sqcap$ ,  $\sqcup$ ],
  opr  $\preceq$ , opr  $\sqcap$ , opr  $\sqcup$ ]
  extends { PartialOrderAndLattice[T,  $\preceq$ ,  $\sqcap$ ,  $\sqcup$ ],
    MeetBoundedLattice[T,  $\sqcap$ ,  $\sqcup$ ],
    HasMinimalElement[T,  $\preceq$ ] }
  where { T coerces Identity[ $\sqcap$ ], T coerces MaximalElement[ $\preceq$ ] }
  property T.coercion (Identity[ $\sqcap$ ]) = T.coercion (MaximalElement[ $\preceq$ ])
end
```

The partial order associated with a meet-bounded lattice should have a maximal element, which is the identity of the meet operator.

```
trait PartialOrderAndJoinBoundedLattice[T extends PartialOrderAndJoinBoundedLattice[T,  $\preceq$ ,  $\sqcap$ ,  $\sqcup$ ],
  opr  $\preceq$ , opr  $\sqcap$ , opr  $\sqcup$ ]
  extends { PartialOrderAndLattice[T,  $\preceq$ ,  $\sqcap$ ,  $\sqcup$ ],
    JoinBoundedLattice[T,  $\sqcap$ ,  $\sqcup$ ],
    HasMaximalElement[T,  $\preceq$ ] }
  where { T coerces Identity[ $\sqcup$ ], T coerces MinimalElement[ $\preceq$ ] }
  property T.coercion (Identity[ $\sqcup$ ]) = T.coercion (MinimalElement[ $\preceq$ ])
end
```

The partial order associated with a join-bounded lattice should have a minimal element, which is the identity of the join operator.

```
trait PartialOrderAndBoundedLattice[T extends PartialOrderAndBoundedLattice[T,  $\preceq$ ,  $\sqcap$ ,  $\sqcup$ ],
  opr  $\preceq$ , opr  $\sqcap$ , opr  $\sqcup$ ]
  extends { PartialOrderAndMeetBoundedLattice[T,  $\preceq$ ,  $\sqcap$ ,  $\sqcup$ ],
    PartialOrderAndJoinBoundedLattice[T,  $\preceq$ ,  $\sqcap$ ,  $\sqcup$ ] }
end
```

The partial order associated with a bounded lattice should have both a minimal element and a maximal element.

37.5 Monoids, Groups, Rings, and Fields

```
trait ApproximateMonoid[T extends ApproximateMonoid[T,  $\odot$ ,  $\approx$ ], opr  $\odot$ , opr  $\approx$ ]
  extends { ApproximatelyAssociative[T,  $\odot$ ,  $\approx$ ], HasIdentity[T,  $\odot$ ] }
  where { T coerces Identity[ $\odot$ ] }
end
```

An *approximate monoid* is a set of values with an approximately associative binary operator $\odot$ that has an identity. For example, floating-point multiplication has identity 1 and is approximately associative.

```
trait Monoid[T extends Monoid[T,  $\odot$ ], opr  $\odot$ ]
  extends { ApproximateMonoid[T,  $\odot$ , =], Associative[T,  $\odot$ ] }
  where { T coerces Identity[ $\odot$ ] }
end
```

A *monoid* is a set of values with an associative binary operator $\odot$ that has an identity. For example, trait `String` extends `Monoid[String,  $\|$ ]` where $\|$ is the string concatenation operator. Note that string concatenation is associative but not commutative and that the empty string is the identity for string concatenation, so coercing `Identity[ $\|$ ]` to type `String` produces the empty string.

Polymorphic Symmetric Multiple Dispatch with Variance

PROVEN!

Who Gyunghee Park, Jaemin Hong, Guy L. Steele Jr., Sukyoung Ryu

Track POPL 2019 Research Papers

$$\begin{array}{c}
 \text{Well-formed method declarations: } \Delta; \Gamma; \overline{V} \overline{P} \vdash \\
 \hline
 \Delta \vdash \overline{\{\xi\}} <: Q <: \{\xi\} \text{ ok} \quad \Delta' = \Delta \cup \{\overline{\{\xi\}} <: \{\xi\}\} \\
 \text{distinct}(\overline{x}) \quad \overline{\Delta'} \vdash \tau \text{ ok} \quad \Delta' \vdash \omega \text{ ok} \\
 \text{distinct}(\overline{P}, \overline{Q}) \quad (\overline{V} = \Rightarrow) \vee (\forall \mathcal{T}. (((\overline{\tau}) \rightarrow \omega) \neq \mathcal{T} [P]) \vee (\Delta \vdash \mathcal{T} \text{ variance } V)) \\
 \hline
 \Delta; \Gamma; \overline{V} \overline{P} \vdash m[\overline{\{\xi\}} <: Q <: \{\xi\}] (\overline{x} : \overline{\tau}). \omega = e \text{ ok}
 \end{array}$$

Fig. 3. Well-formed programs, class declarations, and method declarations

$$\begin{array}{c}
 \text{Return Type Rule: } \Delta \vdash d \text{ return type wrt } d \text{ ok} \\
 \hline
 \text{arrow}(d) = \forall [\overline{\kappa}] (\alpha \rightarrow \rho) \quad \overline{\kappa} = \{\overline{\chi}\} <: P <: \{\overline{\eta}\} \quad \text{arrow}(d') = \\
 \overline{\kappa'} = \{\overline{\chi'}\} <: Q <: \{\overline{\eta'}\} \quad \text{distinct}(\overline{P}, \overline{Q}) \quad \Delta \vdash \text{dom}(d) \\
 \Delta \vdash \forall [\overline{\kappa}] (\alpha \rightarrow \rho) \subseteq \forall [\overline{\kappa}, \overline{\kappa'}] ((\alpha \sqcap \alpha') \rightarrow \rho') \\
 \hline
 \Delta \vdash d \text{ return type wrt } d' \text{ ok}
 \end{array}$$

Fig. 5. Overloading rules for FG

$$\begin{array}{c}
 \text{Universal inner subtyping: } \Delta \vdash Y \lesssim Y \text{ using } \sigma \\
 \hline
 \Delta' = \Delta \cup \{\{\overline{\chi'}\} <: Q <: \{\overline{\eta'}\}\} \quad \{\overline{Q}\} \cap FV(\overline{\chi}, \overline{\eta}, \alpha) = \emptyset \quad \sigma = \left[\frac{\overline{\gamma}/\overline{Q}}{\overline{\gamma}} \right] \quad \overline{\gamma} \neq \text{Bottom} \\
 \Delta' \vdash \sigma \text{ on } \langle \overline{P} \rangle \text{ obeys } \langle \overline{\{\chi\}} \rangle \text{ and } \langle \overline{\{\eta\}} \rangle \quad \Delta' \vdash \sigma \alpha <: \alpha' \\
 \hline
 \Delta \vdash \forall [\overline{\{\chi\}} <: P <: \{\overline{\eta}\}] \alpha \lesssim \forall [\overline{\{\chi'}\} <: Q <: \{\overline{\eta'}\}] \alpha' \text{ using } \sigma
 \end{array}$$

Fig. 6. Subtype relations of quantified types

Coherence can be bypassed by an indirect impl for a trait object #57893

New issue

Open

arielb1 opened this issue on Jan 25, 2019 · 50 comments



arielb1 commented on Jan 25, 2019 · edited by lcnr

Contributor



Comments

The check for manual `impl Object for Object` only makes sure there is no *direct* `impl Object for dyn Object` - it does not consider such indirect impls. Therefore, you can write a blanket `impl<T: ?Sized> Object for T` that conflicts with the builtin `impl Object for dyn Object`.

Assignees



nikomatsakis

Labels

A-dst

A-traits

C-bug

I-unsound

P-high

T-compiler

T-lang

T-types

Formalizing Rust Traits



- Mater Thesis by Jonatan Milewski (Ron Garcia)
 - ▶ NanoRust: a language with traits
 - ▶ MicroRust: NanoRust with associated types
 - ▶ MiniRust: MicroRust with trait objects
 - ▶ RustIn: an internal language for MiniRust
 - ▶ Translating MiniRust programs to RustIn

Formalizing Rust Traits

by

Jonatan Milewski

B. Sc., McGill University, 2013

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF

Master of Science

in

THE FACULTY OF GRADUATE AND POSTDOCTORAL STUDIES
(Computer Science)

The University of British Columbia
(Vancouver)

November 2015

© Jonatan Milewski, 2015

<https://open.library.ubc.ca/media/stream/pdf/24/1.0220521/4>

NanoRust: Well-Typed Programs and Items

$\boxed{\vdash_P \text{pgm} : \Gamma \mid \Theta}$ (Well-typed programs)

$$\text{(pgm)} \frac{\overline{\Gamma \mid \Theta \vdash \text{item}_i : \Gamma_i \mid \Theta_i} \quad \Gamma = \overline{\Gamma_i} \quad \Theta = \overline{\Theta_i}}{\vdash_P \overline{\text{item}_i} : \Gamma \mid \Theta}$$

$\boxed{\Gamma \mid \Theta \vdash \text{item} : \Gamma \mid \Theta}$ (Well-typed items)

$$\begin{aligned} \text{(struct)} \frac{\overline{\Gamma, \overline{T_j} \vdash_{\text{WF}} \tau_i}^i}{\Gamma \mid \Theta \vdash \text{struct } S \langle \overline{T_j} \rangle \{ \overline{x_i : \tau_i} \} : [S \langle \overline{T_j} \rangle \{ \overline{x_i : \tau_i} \}] \mid \emptyset} \\ \text{(fun)} \frac{\sigma = \forall \overline{T_k}. \overline{\pi_j} \Rightarrow \text{fn}(\overline{\tau_i}) \rightarrow \tau \quad \Gamma \vdash_{\text{WF}} \sigma \quad \Gamma, \overline{T_k}, \overline{x_i : \tau_i} \mid \Theta, \overline{\pi_j} \vdash e : \tau}{\Gamma \mid \Theta \vdash \text{fn } f \langle \overline{T_k} \rangle (\overline{x_i : \tau_i}) \rightarrow \tau \text{ where } \overline{\pi_j} \{ e \} : [f : \sigma] \mid \emptyset} \end{aligned}$$

NanoRust in SpecTec

```
;; Since we need mutual context for all items,  
;; we first collect all contexts w/o type checking.  
;; Then, we type check with those contexts.
```

```
;; (Well-typed programs)
```

```
rule Program_ok/pgm:
```

```
|- pgm : typeEnv ~~ consEnv
```

```
-- if item* = pgm
```

```
-- Construct_pgm_context: item* ~> typeEnv ~~ consEnv
```

```
-- (Item_ok: typeEnv ~~ consEnv |- item)*
```

```
;; (Well-typed items)
```

```
rule Item_ok/struct:
```

```
typeEnv ~~ consEnv |- STRUCT id tid* { varType* }
```

```
-- if typeEnv' = typeEnv ++ (TVAR tid)*
```

```
-- if (_ ': type = varType)*
```

```
-- (Scheme_wf: typeEnv' |- (JUST type))*
```

```
rule Item_ok/func:
```

```
typeEnv ~~ consEnv |- FN id tid* varType* -> type WHERE cons* { term }
```

```
-- if (_ : t = varType)*
```

```
-- if funType = FUNCT t* type
```

```
-- if scheme = FORALL tid* . cons* -> (JUST funType)
```

```
-- Scheme_wf: typeEnv |- scheme
```

```
-- if typeEnv' = typeEnv ++ (TVAR tid)* ++ ($varTypeToTypeEnv(varType))*
```

```
-- if consEnv' = consEnv ++ (JUST cons)*
```

```
-- Term_ok: typeEnv' ~~ consEnv' |- term : type
```

An Empirical Study of Bugs in the rustc Compiler

Authors:  Zixi Liu,  Yang Feng,  Yunbo Ni,  Shaohua Li,  Xizhe Yin,  Qingkai Shi,  Baowen Xu,  Zhendong Su | [Authors Info & Claims](#)

Proceedings of the ACM on Programming Languages, Volume 9, Issue OOPSLA2 • Article No.: 411, Pages 3869 - 3896
<https://doi.org/10.1145/3763800>

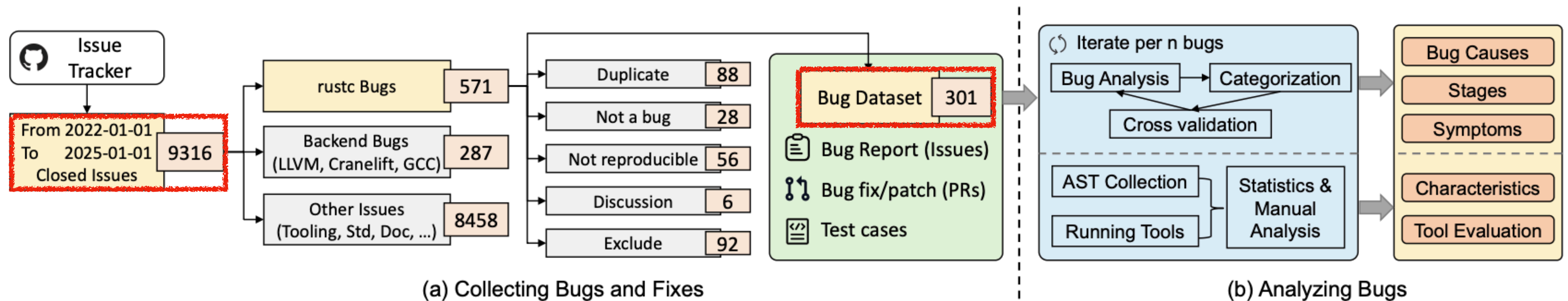


Fig. 2. The overview of our bug collection and analysis process.

Table 1. Bug labels and corresponding descriptions in Rust’s official GitHub issue tracker [rust team 2025b].

Category	Label	# Num	Description
HIR	A-HIR	20	The high-level intermediate representation (HIR)
	A-THIR	1	Typed HIR
MIR	A-MIR	43	Mid-level IR (MIR) - https://blog.rust-lang.org/2016/04/19/MIR.html
	A-mir-opt	78	MIR optimizations
	A-mir-opt-inlining	23	MIR inlining
	A-mir-opt-GVN	0	MIR opt Global Value Numbering (GVN)
	A-mir-opt-nrvo	0	Fixed by the Named Return Value Opt. (NRVO)
	A-stable-MIR	1	stable MIR
Type	A-type-system	25	Type system
	A-inference	29	Type inference
	A-closures	29	Closures (<code> ... { ... }</code>)
	A-coercions	13	implicit and explicit <code>expr as Type</code> coercions
	A-const-generics	70	const generics (parameters and arguments)
	A-DSTs	0	Dynamically-sized types (DSTs)
	A-zst	0	Zero-sized types (ZST).
	A-trait-system	77	Trait system
	A-impl-trait	68	Universally/existentially quantified anonymous types with static dispatch
	A-trait-objects	27	trait objects, vtable layout
	A-auto-traits	14	auto traits (e.g., <code>auto trait Send {}</code>)
	A-implied-bounds	9	Implied bounds / inferred outlives-bounds
	A-coinduction	0	Concerning coinduction, most often for auto traits
	A-coherence	14	Coherence
Lifetimes	A-lifetimes	70	Lifetimes / regions
	A-borrow-checker	45	The borrow checker
Backend (Excluded)	A-LLVM	275	Code generation parts specific to LLVM.
	A-gcc	2	Things relevant to the [future] GCC backend
	A-cranelfit	10	Things relevant to the [future] cranelift backend

Table 3. The taxonomy of bug causes.

Category	Subcategory	Description	# Bugs	Ratio
Type System Errors	Trait & Bound	The errors were caused by rustc's handling of traits and its enforcement of type parameter constraints, such as requiring specific traits or conditions.	37	12.3%
	Opaque types	The errors were caused by issues within rustc's handling of opaque types, which rely on the ownership system, zero-cost abstractions, and the design of generics and traits.	38	12.6%
	New solver	The errors are caused due to the interaction between rustc's new solver, which is designed to improve trait-bound resolution and reduce workload, and the existing old solver.	7	2.3%
	Well-formedness	The errors were caused by rustc's well-formedness checking, including ownership, lifetime, type system, and the borrow checker.	9	3.0%
	Subtotal	-	91	30.2%
Ownership & Lifetime Errors	Borrow & Move	The errors were caused by issues in implementing the ownership model, which ensures memory safety and concurrency safety through the move and borrow semantics.	7	2.3%
	Lifetime	The errors were caused by issues in rustc's lifetime checking, which ensures that every reference is valid and does not outlive the data it points to.	34	11.3%
	Subtotal	-	41	13.6%
MIR Optimization Errors	Wrong implementations	The errors were caused by incorrect implementations of rustc's MIR-based optimizations (e.g., constant propagation, dead code elimination, inlining).	34	11.3%
	Missing cases	Some specific corner cases of the optimization algorithm were not considered thoroughly.	12	4.0%
	Subtotal	-	46	15.3%
General Errors	Basic structure	Bugs caused by rustc errors in processing features like closures and internal data structures.	38	12.6%
	Error handling & Reporting	The errors were caused by rustc's failure to handle exceptional cases properly or its misprocessing of reports, leading to misleading error messages or incorrect error locations.	75	24.9%
	Compatibility	The bugs were triggered by certain operating systems, bugs in the back-end LLVM, or errors specific to the Rust edition.	10	3.3%
	Subtotal	-	123	40.9%

Table 4. Distribution of bug symptoms and the distribution of bug symptoms per cause.

Symptoms		Occurrence		Type System	Ownership & Lifetime	MIR Optimization	General Errors
Crash	Front-end Panic (valid)	42 (14.0%)	120 (39.9%)	30 (25.0%)	3 (2.5%)	19 (15.8%)	68 (56.7%)
	Front-end Panic (invalid)	75 (24.9%)					
	Back-end Crash	3 (1.0%)					
Correctness Issues	Completeness Issues	56 (18.6%)	78 (25.9%)	43 (55.1%)	17 (21.8%)	7 (9.0%)	11 (14.1%)
	Soundness Issues	22 (7.3%)					
Miscompilation	Inconsistent Output Issues	18 (6.0%)	30 (10.0%)	5 (16.7%)	4 (13.3%)	12 (40.0%)	9 (30.0%)
	Safe Rust Causes UB	12 (4.0%)					
Diagnostic Issues	Incorrect Warning/Error	20 (6.6%)	58 (19.3%)	12 (20.7%)	16 (27.6%)	1 (1.7%)	29 (50.0%)
	Improper Fix Suggestion	38 (12.6%)					
Misoptimization	Incorrect Optimization	9 (3.0%)	15 (5.0%)	1 (6.7%)	1 (6.7%)	7 (46.7%)	6 (40.0%)
	Performance Issues	6 (2.0%)					

Soundness Issues: Soundness bugs refer to situations where rustc mistakenly accepts programs that should be rejected due to violating language rules. Rust is known for its strict rules around syntax and semantics, and soundness bugs occur when rustc incorrectly allows code that violates these rules to compile successfully.

Collecting Soundness Issues in `rustc`

- `rustc`'s soundness issues reported between 2022/01/01 and 2025/09/01
 - ▶ Including the two most recent editions of the language:
Rust 2021 (released on 2021/10/21) & Rust 2024 (released on 2025/02/20)
 - ▶ 969 issues with the 14 type-related labels by the previous work (Table 1)
 - ▶ 320 issues marked with either C-bug or I-unsound
 - ▶ 27 soundness issues: 1) containing a Rust code snippet that the reporter claimed the compiler should reject, and 2) the buggy behavior was caused by the compiler (manual inspection)
 - ▶ 23 unique soundness issues

Table 4. Soundness issues in rustc

Issue #	Affected Feature	Symptom	Consequence	Triggers	Status
96927	Type inference	Type argument	Inconsistency	try	Confirmed
98543	Lifetimes	Missing lifetime checks	Memory bug	implied, assoc	Fixed
99158 [†]	Lifetimes	Missing lifetime checks	Memory bug	implied, fn ptr	Confirmed
99554	Traits	Orphan	Breaking change	assoc	Fixed*
100051	Lifetimes	Missing lifetime checks	Memory bug	implied, assoc, HR	Reported
105251	Lifetimes	Missing lifetime checks	Inconsistency	assoc, RPIT	Fixed
105787	Traits	Overlapping	Inconsistency	assoc, HR	Fixed
106040	Traits	Incorrect solving	Inconsistency	assoc	Reported
107268	Type inference	Type argument	Inconsistency	assoc, fn ptr	Fixed
108468	Lifetimes	Anonymous	Inconsistency	IT, async	Reported
109628	Lifetimes	Missing lifetime checks	Inconsistency	assoc	Fixed
110925	Type wf-ness	Missing wf-ness checks	Inconsistency	RPIT	Reported
111935	Type inference	Hidden type	Inconsistency	RPIT, rec	Fixed
112417	Lifetimes	Missing lifetime checks	Memory bug	implied, RPIT	Fixed
114061	Traits	Overlapping	Inconsistency	assoc, HR	Fixed
114389 [‡]	Traits	Overlapping	Memory bug	assoc, tr obj	Confirmed
123312	Lifetimes	Missing lifetime checks	Inconsistency	clo	Reported
133361	Traits	Overlapping	Memory bug	assoc, tr obj	Fixed
134311	Type inference	Hidden type	Inconsistency	assoc, RPIT	Reported
136508	Type cast	Incorrect cast	Misleading	enum	Reported
139406	Type inference	Hidden type	Inconsistency	RPIT, rec	Fixed
141713	Lifetimes	Missing lifetime checks	Memory bug	assoc, HR, clo	Confirmed
144045	Lifetimes	Missing lifetime checks	Inconsistency	arr	Reported

[†] A duplicate of Issue 25860, which was reported in 2015.

[‡] A duplicate of Issue 57893, which was reported in 2019.

* A warning occurs instead of an error.

Table 1. Classification of affected features and symptoms of soundness issues

Affected feature	Symptoms
Lifetimes	Missing lifetime checks, Anonymous lifetimes allowed
Traits	Overlapping implementations, Orphan implementations, Incorrect trait solving
Type inference	Arbitrary type argument selection, Arbitrary hidden type selection
Type well-formedness	Missing well-formedness checks
Type cast	Incorrect type cast allowed

Table 2. Classification of consequences of soundness issues

Consequence	Description
Memory bug	A memory bug can occur at run time
Breaking change	A benign change in a library can lead to a compilation error in a client code
Inconsistency	A code snippet passes type checking while similar ones are rejected
Misleading behavior	The run-time behavior could be misleading

Table 3. Classification of status of soundness issues

Status	Description
Fixed	The compiler has been fixed, and the code snippet is rejected by the latest release (1.93.0)
Confirmed	The C-bug or I-unsound label has been assigned to by a person other than the reporter
Reported	The issue is open but not fixed or confirmed yet
Non-bug	The issue has been closed without being fixed or confirmed

RQ1. Symptoms and Consequences

- Finding 1. The Rust compiler has soundness issues that lead to memory bugs, caused by missing lifetime checks or overlapping `impls`.
- Finding 2. Rust programmers often believe that the compiler should ensure consistency by rejecting even programs that do not incur memory bugs.

Table 5. Symptoms and consequences of soundness issues

			Consequence				Total	
			Memory	Breaking	Inconsistency	Misleading		
Symptom	Lifetimes	Missing lifetime checks	5	0	4	0	9	10
		Anonymous lifetimes	0	0	1	0	1	
	Traits	Overlapping impls	2	0	2	0	4	6
		Orphan impls	0	1	0	0	1	
		Incorrect trait solving	0	0	1	0	1	
	Inference	Type argument	0	0	2	0	2	5
		Hidden type	0	0	3	0	3	
	Wf-ness	Missing wf-ness checks	0	0	1	0	1	1
	Cast	Incorrect cast	0	0	0	1	1	1
Total			7	1	14	1	23	

RQ2. Triggering Features

- Finding 3. Compiler developers should carefully handle associated types wherever they can occur.
- Finding 4. The interaction between orthogonal features, especially lifetimes and traits, makes it difficult for type checking to be sound.
- Finding 5. Correct treatment of implied bounds and trait objects is extremely important in ensuring memory safety.

Table 6. Triggering features of soundness issues

		Triggering Feature														Total
		implied	assoc	IT	RPIT	tr obj	HR	fn ptr	clo	rec	async	try	arr	enum		
Affected	Lifetimes	4	5	1	2	0	2	1	2	0	1	0	1	0	10	
	Traits	0	6	0	0	2	2	0	0	0	0	0	0	0	6	
	Inference	0	2	0	3	0	0	1	0	2	0	1	0	0	5	
	Wf-ness	0	0	0	1	0	0	0	0	0	0	0	0	0	1	
	Cast	0	0	0	0	0	0	0	0	0	0	0	0	1	1	
Consequence	Memory	4	5	0	1	2	2	1	1	0	0	0	0	0	7	
	Breaking	0	1	0	0	0	0	0	0	0	0	0	0	0	1	
	Inconsistency	0	7	1	5	0	2	1	1	2	1	1	1	0	14	
	Misleading	0	0	0	0	0	0	0	0	0	0	0	0	1	1	
Total		4	13	1	6	2	4	2	2	2	1	1	1	1	23	

RQ3. Bug Consensus

- Finding 6. The `rustc` developers are inclusive in discussing soundness issues even when they do not lead to memory bugs.

Table 7. Status of soundness issues

		Status				Total
		Fixed	Confirmed	Reported	Non-bug	
Consequence	Memory	3	3	1	0	7
	Breaking	1	0	0	0	1
	Inconsistency	7	1	6	0	14
	Misleading	0	0	1	0	1
Total		11	4	8	0	23

RQ4. Issue Lifecycles

- Finding 7. Soundness issues are typically discovered long after they are introduced, persisting across multiple releases.
- Finding 8. New features are often stabilized before undergoing sufficient testing.

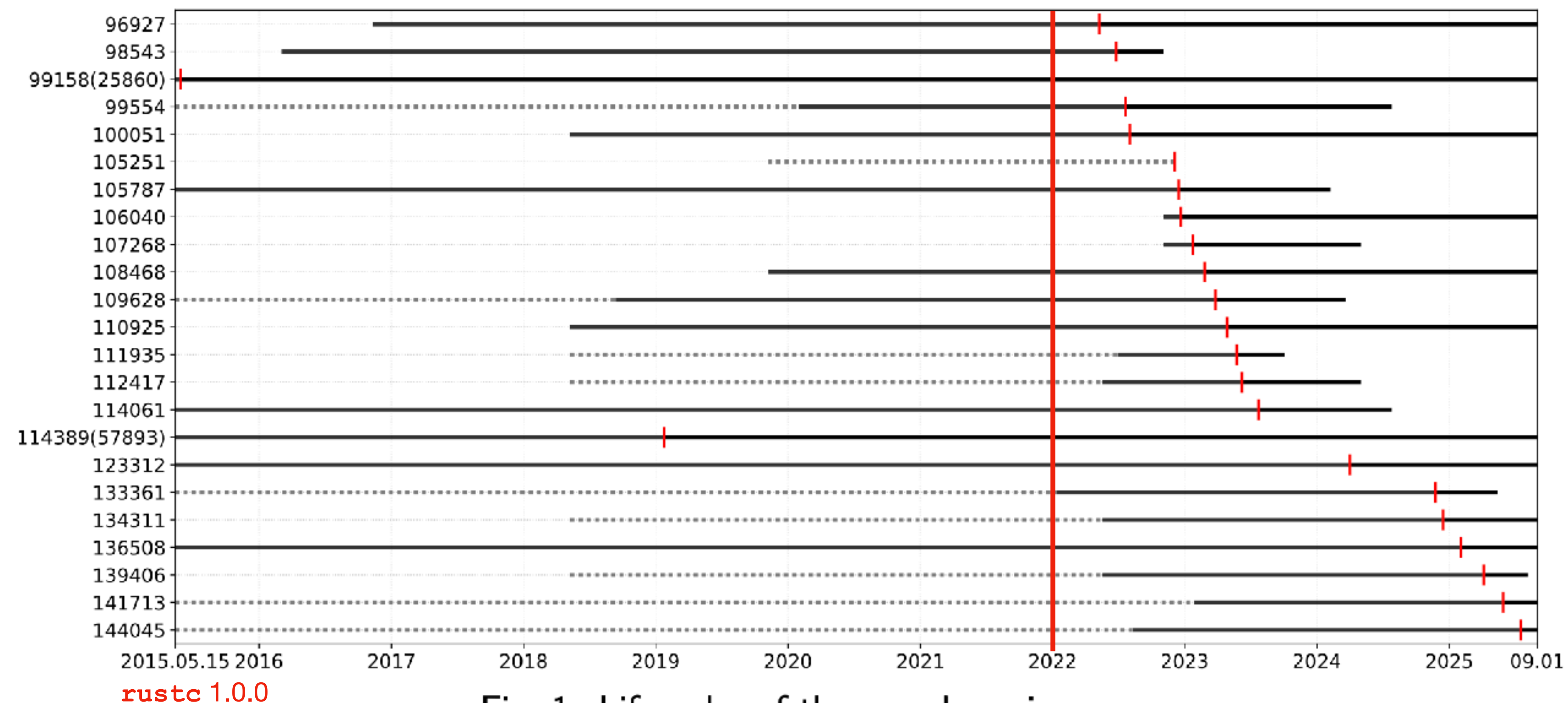


Fig. 1. Lifecycles of the soundness issues

Potential Oracles: Implementations

- Miri <https://github.com/rust-lang/miri>
 - ▶ A Rust interpreter designed to detect UB, including memory bugs
 - ▶ Cannot identify soundness bugs that do not manifest as UB
- Chalk <https://github.com/rust-lang/chalk>
 - ▶ An implementation of the Rust trait system based on a logic solver.
 - ▶ Targets only the trait system; other type features remain outside its scope
- a-mir-formality <https://github.com/rust-lang/a-mir-formality>
 - ▶ An executable formal model of the Rust type system
 - ▶ Supports only a limited subset of Rust

Potential Oracles: Documentation

- Rust Reference
 - ▶ An official Rust project
- FLS
 - ▶ Developed by Ferrous Systems and AdaCore for use in safety-critical industries
 - ▶ Adopted as a Rust project in 2025
- Rust RFCs
 - ▶ A collection of proposed changes to the language
 - ▶ Certain features are explained in detail only within RFCs

Potential Oracles: Documentation

- Implied bounds and well-formedness
 - ▶ Rust Reference: “lifetime bounds required for types to be well-formed are **sometimes inferred**”
 - ▶ FLS: “an implied bound is ... **the byproduct of relations** between lifetime parameters and function parameters ...”
 - ▶ Rust RFCs
 - The definition of well-formedness exists only in RFCs, notably RFC 1214.
 - This RFC defines well-formedness using inference rules based on its own syntax for Rust types. Unfortunately, this syntax lacks several types present in Rust, most notably `impl` trait types.

Rust's Type Checker Implementation is Unsound: An Empirical Study on Soundness Bugs in rustc

Who *Yusung Sim, Sukyoung Ryu, Jaemin Hong*

Track [ISSTA 2026 Research papers](#)

Abstract Rust is claimed to be a *type-sound* language capable of preventing various undesirable behaviors, including memory bugs. However, `rustc`, the official Rust compiler, is not immune to defects; it contains *soundness bugs*, where the compiler accepts programs that should be rejected during type checking. In this work, we present an empirical study of 23 issues that report potential soundness bugs in `rustc`, collected from the GitHub issue tracker between January 1, 2022 and September 1, 2025. We analyze each issue in depth, focusing on its *affected feature*, *symptom* (how the feature is mishandled), *consequence* (the resulting undesirable behavior), *triggering features*, *community consensus* regarding whether it is a bug, and *lifecycle*, including introduction, discovery, and fix. Furthermore, we investigate existing artifacts, including implementations such as Miri, Chalk, and a-mir-formality, alongside documentation such as the Rust Reference, the FLS, and Rust RFCs to assess their potential as oracles for testing the type soundness of `rustc`. Our key findings indicate that: (1) Certain soundness bugs, typically triggered by *implied bounds* or *trait objects*, compromise memory safety. (2) Sound type checking is challenged by edge cases involving *associated types* and the interaction between *lifetimes* and *traits*. (3) Most bugs persist from the initial introduction of the relevant features and require significant time to be discovered. (4) While Miri can detect soundness bugs that lead to memory bugs, a-mir-formality and Chalk are currently immature despite their potential to identify other bug categories. (5) Existing documentation frequently fails to provide precise explanations of the language semantics.



Yusung Sim
KAIST



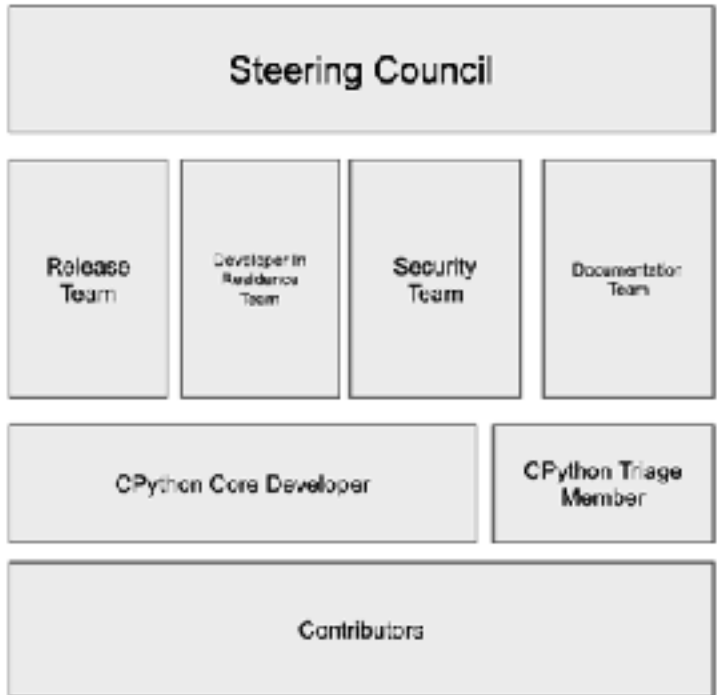
Sukyoung Ryu
KAIST
South Korea



Jaemin Hong
UNIST
South Korea

Introduction to the Python Core Team

- Triage Member
- Core Developer
- Release Manager
- Steering Council Member



2026 Steering Council Member

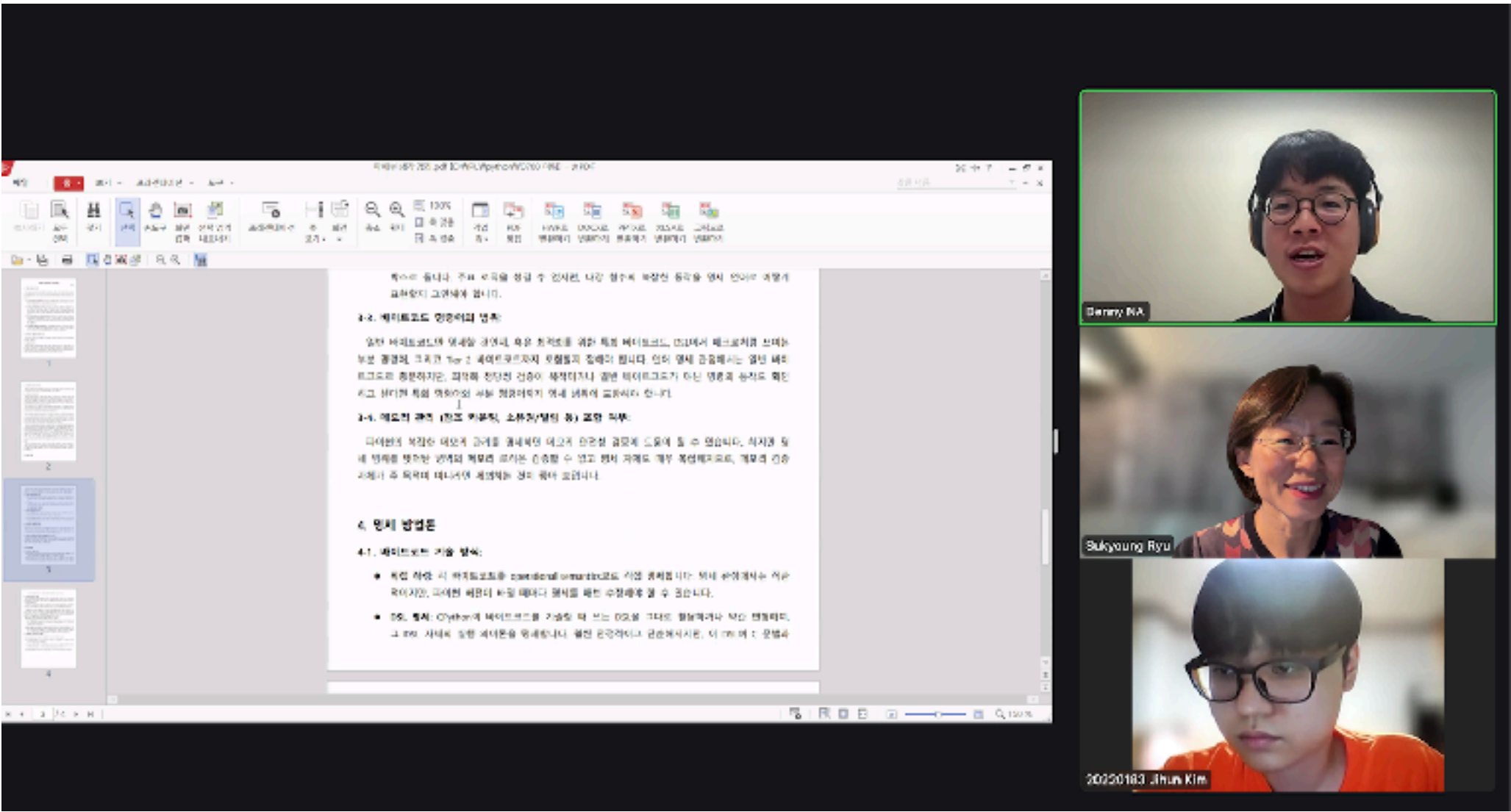
- Pablo Galindo Salgado: Hudson River Trading
- Savannah Ostrowski: FastAPI Labs
- Barry Warsaw: Nvidia
- Donghee Na: Karrot
- Thomas Wouters: Meta



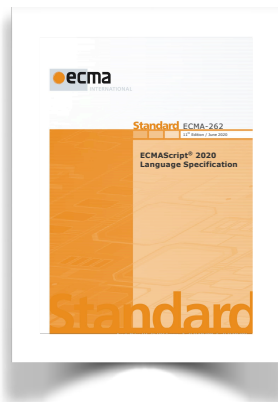
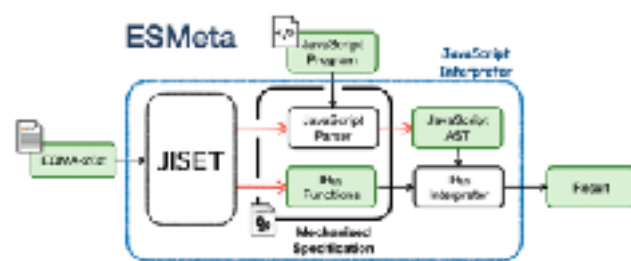
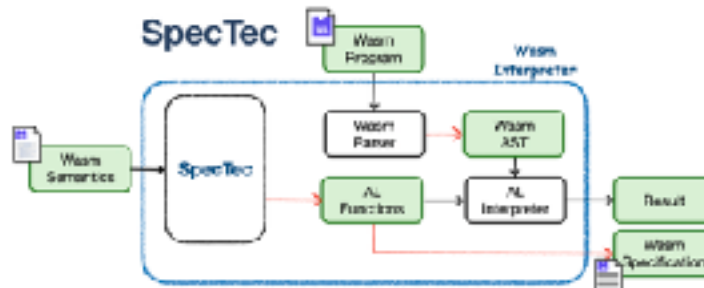
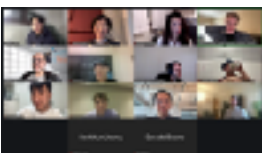
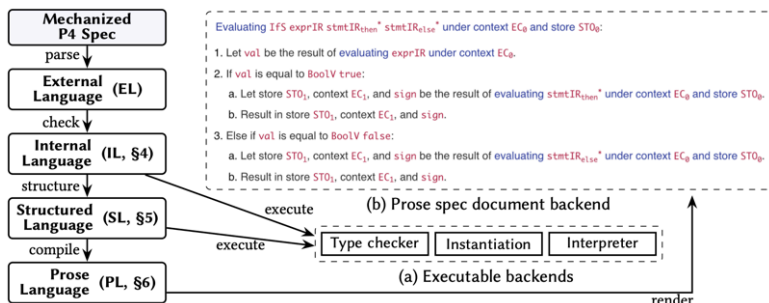
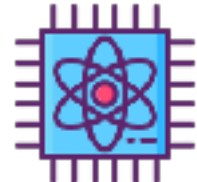
<https://peps.python.org/pep-8107/>

Steering Council Responsibilities

- Accept/reject PEPs (Python Enhancement Proposals)
- Maintain quality & stability of language & CPython
- Work with the PSF to manage project assets
- Seek consensus
- Delegate decisions whenever possible
- Veto (or not!) newly elected core members
- Office Hours



ESMeta, SpecTec, P4-SpecTec, and More

	Problems	Requirement	Solution	Burden	Community & Researchers
	Bugs in the specification and diverse implementations		Type checking of the specification and rich test generation	ESMeta team's maintenance	  
	Bugs and inconsistencies across artifacts	1. Formal Specification 2. Prose Specification 3. Reference Interpreter 4. Test Suite 	Automatic generation of the artifacts	Community's learning a specification language	  
	No specification	P4c	Understanding, discussing, specifying, and testing the semantics	Community's learning a specification language	  
More	        	           			

SpecTrum: Specification-Guided Differential Fuzzing for Ethereum Consensus Clients

<https://arxiv.org/abs/2608.17738>

Who
Seokhun Jeong, Gyeongmin Dan, Sukyoung Ryu, Sungjae Hwang

Track
ASE 2026 Research Papers



Seokhun Jeong
KAIST
South Korea



Gyeongmin Dan
Sungkyunkwan University
South Korea



Sukyoung Ryu
KAIST
South Korea



Sungjae Hwang
Sungkyunkwan University

SPECTRUM: Specification-Guided Differential Fuzzing for Ethereum Consensus Clients

ASE '26, October 12–16, 2026, Munich, Germany

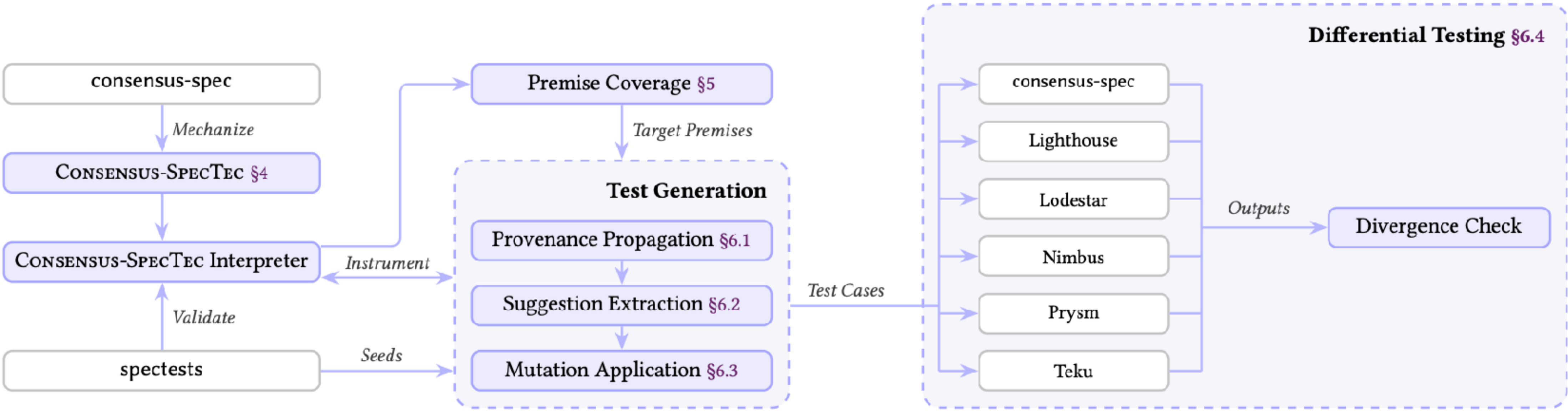
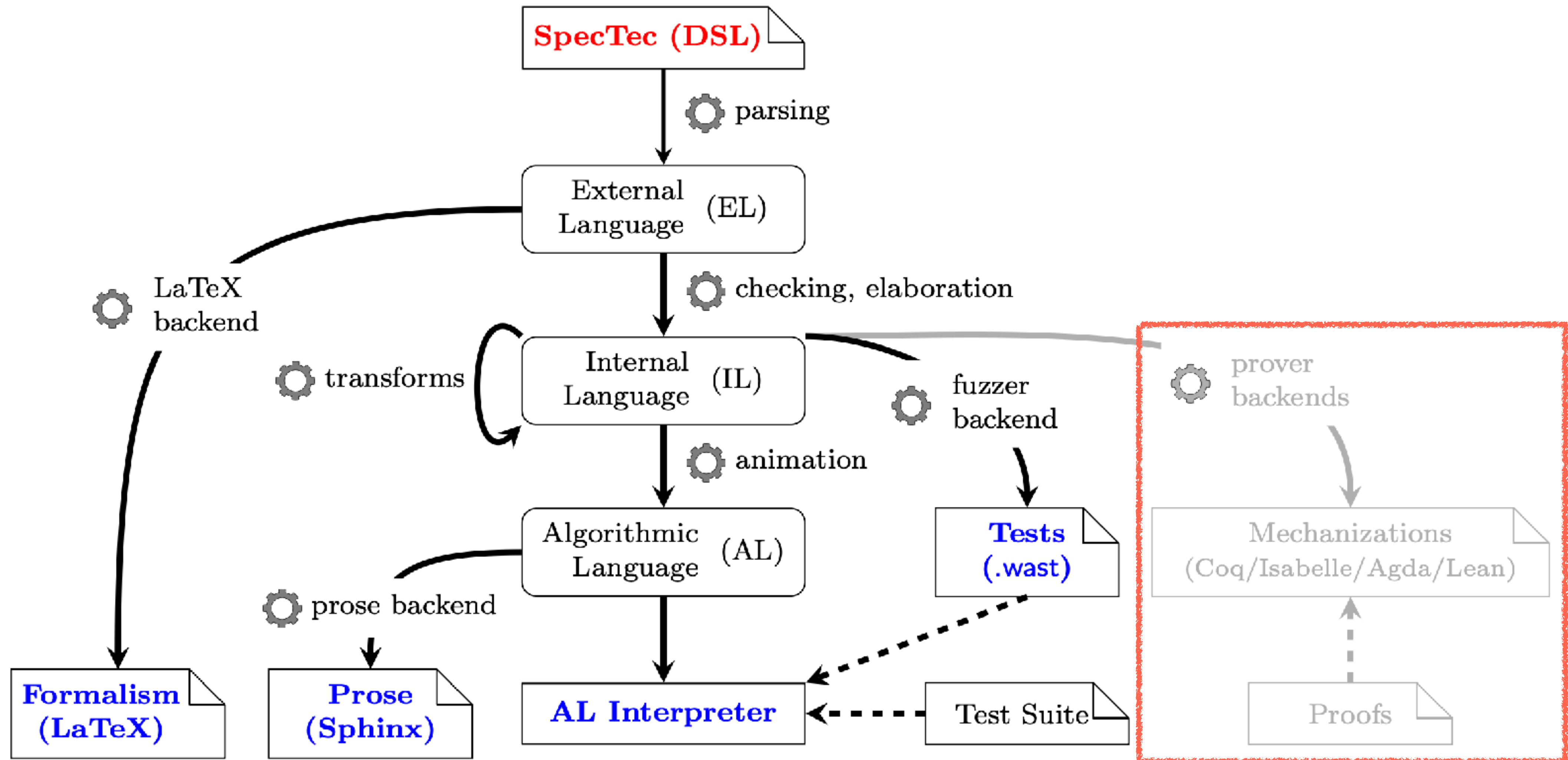


Figure 1: Overview of the SPECTRUM framework.

Mechanized Specifications as an Enabler



Mechanized Specifications as an Enabler

Program Verification Workshop 2026

July 28–30, 2026

<https://sf.snu.ac.kr/cris/camp-2026/>



PLDI 2023

Plenary Panel 

Reflecting on 50 Years of Computing Research, and Future Outlook

*Hagit Attiya, Technion | Jack Dongarra, University of Tennessee at Knoxville
Mary Hall, University of Utah | Lizy Kurian John, The University of Texas at Austin
Huan Liu, Arizona State University | Guy L. Steele Jr., Oracle Labs*

**Tuesday, June 20
4:15 - 5:15 PM ET**

YOUTUBE.COM
FCRC Plenary Panel: Reflecting on 50 Years of Computing Research, & Future Outlook



- "If you go and talk to an old person and he or she says, "We tried that 25 years ago and it didn't work," don't take that for an answer, ask why it didn't work and what's different today."
- "Keep the big picture in mind; look for opportunities to bridge disciplines rather than rat holding on just one; don't settle for just a 5% improvement, aim big; and finally **read read read** voraciously, read papers, read code, read books, absorb as much information as you can, you never can tell what you learned five years ago might suddenly be relevant now."